Министерство инноваций, цифрового развития и инфокоммуникационных технологий Республики Саха (Якутия)
ГАПОУ РС(Я) «Якутский колледж связи и информационных технологий имени П.И. Дудкина»

«Допустить к защите»
Зам. директора по УР
_____А. Н. Филиппович
«___»_____20__ г.

КУРСОВАЯ РАБОТА

по дисциплине: МДК 01.03 Разработка мобильных приложений

тема: «Разработка мобильного приложения GameKritik»

Специальность 09.02.07 Информационные системы и программирование

Студент: _____ В. В. Спиридонов
подпись, дата

Руководитель: _____ Н. И. Фёдоров
подпись, дата

Якутск 2026

СОДЕРЖАНИЕ

Введение 3

1 Проектирование мобильного приложения 4

1.1 Описание предметной области 4

1.2 Анализ существующих решений 4

1.3 Требования к приложению 5

1.4 Выбор стека технологий 7

1.5 Архитектура приложения 8

1.6 Проектирование интерфейса в Figma 8

1.7 Проектирование базы данных в Supabase 10

2 Реализация мобильного приложения 11

2.1 Настройка среды разработки и подключение Supabase 11

2.2 Реализация ключевых экранов приложения 12

2.3 Взаимодействие с базой данных Supabase 16

2.4 Реализация дополнительного функционала 18

2.5 Работа с Git 20

Заключение 21

Список использованных источников 22

Приложение А. Проверка каталога в базе данных Supabase 23

ВВЕДЕНИЕ

Мобильное приложение позволяет сравнивать игры, проверять даты выхода, смотреть трейлеры и читать отзывы на смартфоне. Сейчас эти сведения разделены между магазинами, видеохостингами, форумами и сайтами с руководствами. Поэтому актуальна разработка приложения, объединяющего каталог, материалы и личную активность пользователя.

Тема курсовой работы — проектирование и разработка мобильного приложения GameKritik с удалённой базой данных Supabase. Объектом исследования является процесс предоставления пользователю сведений об играх и обмена отзывами. Предмет исследования — методы построения Android-приложения с каталогом, авторизацией, хранением пользовательских действий и интеграцией внешних игровых материалов. Практический результат работы представляет собой работающее приложение на Kotlin и Jetpack Compose, подключённое к действующему проекту Supabase.

Цель работы — создать Android-приложение для поиска игр, просмотра материалов, публикации отзывов и сохранения личной активности. Для этого проанализирована предметная область; определены требования; спроектированы интерфейс, навигация и база данных; реализованы каталог, регистрация, медиа, гайды, моды, отзывы и профиль; настроены права доступа; выполнена проверка приложения.

При проектировании использовались Figma, язык Kotlin, декларативный интерфейс Jetpack Compose и система Supabase с PostgreSQL и Supabase Auth. Официальная документация Android описывает Compose как инструментарий для создания нативного интерфейса, а документация Supabase объясняет связь аутентификации с управлением доступом к строкам базы данных [1, 6]. В работе рассматривается реализованная версия GameKritik 1.5; сведения о количестве игр и сценариях проверки относятся к состоянию проекта на 21 октября 2026 года.

1 ПРОЕКТИРОВАНИЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ

1.1 Описание предметной области

Предметная область включает цифровой каталог видеоигр и действия пользователя вокруг него. Основная сущность — игра: название, описание, жанры, дата выпуска, разработчик, платформы, оценка, обложка и ссылки на материалы. Игра связана со скриншотами, трейлерами, руководствами и страницами модификаций. Отдельную группу образуют персональные данные: учётная запись, избранные игры, написанные отзывы и история просмотра. Такое разделение позволяет показывать каталог без регистрации, а личные функции открывать после входа.

Содержимое каталога должно быть ограниченным и осмысленным. В текущей версии в Supabase размещено 85 игр известных серий, включая Sonic, Metal Gear, Resident Evil и Persona. Категория «Новые» использует год первого выпуска 2025–2026, а «Скоро выйдут» опирается на статус предстоящего релиза и дату. Такой подход уменьшает нагрузку на пользователя по сравнению с длинным необработанным перечнем и позволяет сохранить понятные разделы навигации. Данные о будущем релизе подлежат обновлению, поскольку сроки выхода могут меняться.

1.2 Анализ существующих решений

В качестве функциональных ориентиров рассмотрены магазины игр, игровые сообщества и информационные порталы. Магазин Steam объединяет страницы продуктов, пользовательские рецензии, изображения и новости. Его сильная сторона — объём данных и активное сообщество, однако интерфейс служит прежде всего экосистеме магазина. Официальный метод Steamworks позволяет получать отзывы с признаками рекомендации и числом отметок «забавно»; эти сведения использованы как внешний источник, а не скопированы в базу приложения [9].

Из сравнения следует набор отличительных решений: единый мобильный каталог без привязки к покупке, краткая страница игры с медиа и

материалами, персональный список избранного и отзывов, история просмотренных проектов и автоматическое соответствие темы системной настройке Android. Приложение не стремится заменить магазин или базу модификаций. Его задача — предоставить компактную точку входа и сохранить контекст пользовательских действий. При разработке поэтому приоритет получили читаемая карточка с названием игры и быстрый переход к нужному разделу.

Таблица 1.1 — Сравнение аналогов

Функция	Steam	Metacritic	GameKritik
Каталог игр	Есть	Есть	Есть
Отзывы игроков	Есть	Свободные оценки	Свои и Steam
Гайды и моды	Сообщество	Нет	Ссылки на источники
История в профиле	Ограниченно	Нет	Есть

1.3 Требования к приложению

Гость должен иметь возможность открыть каталог, выполнить поиск по названию, выбрать жанр и переключиться между популярными, новыми и ожидаемыми играми. Для каждой игры доступны описание, даты, обложка, скриншоты, трейлеры, руководства, моды и отзывы. Полноэкранный просмотр медиа сохраняет пропорции кадра и возвращает пользователя на страницу игры после закрытия. Если источник медиа временно недоступен, интерфейс должен оставаться работоспособным и предлагать переход к официальной странице.

Зарегистрированный пользователь дополнительно сохраняет игру в избранное, ставит оценку по десятибалльной шкале и публикует отзыв. В профиле отображаются ранее написанные отзывы и история просмотренных игр. Повторная отправка отзыва по той же игре обновляет собственную запись, а не создаёт бесконечное число дублей. После перезапуска приложения вход и личные списки восстанавливаются из сохранённой сес-

сии и облачной базы. Просмотр чужих личных списков через клиентский API недопустим.

К нефункциональным требованиям отнесены работа на Android с минимальной поддерживаемой версией, удобное управление на экране телефона, согласованность светлой и тёмной тем, защита токенов и сохранность персональных данных. Пользовательский интерфейс должен оставаться понятным при отсутствии изображения, пустом разделе или сетевой ошибке. Отдельно требуется различать демонстрационную оценку исходного макета, оценку участников GameKritik и внешний показатель Metacritic, чтобы не смешивать источники чисел.

Критериями готовности приняты успешная сборка APK, загрузка каталога из Supabase, регистрация и повторный вход, доступ к защищённым функциям профиля, работа медиа и корректное отображение внешних отзывов Steam. Проверки должны выполняться на эмуляторе, а запросы к реальному облачному проекту — без создания лишних пользовательских аккаунтов в обычных UI-тестах. Эти критерии определяют как структуру кода, так и план проверки во второй главе.

Перед реализацией были выделены три основных маршрута. Первый начинается без регистрации: приветственный экран, каталог, поиск, страница игры, медиа и материалы. Второй маршрут связан с входом и личными действиями: пользователь регистрируется либо авторизуется, добавляет игру в избранное, публикует отзыв и затем находит его в профиле. Третий маршрут начинается с уже сохранённой сессии: приложение восстанавливает доступ, обновляет токен при необходимости и повторно загружает персональные списки. Разделение маршрутов помогает не требовать аккаунт там, где достаточно публичного каталога.

Для каждого маршрута определены неуспешные исходы. Пустой результат поиска сопровождается предложением изменить запрос или жанр. Ошибка каталога позволяет повторить обновление; ошибка авторизации показывает понятную причину рядом с формой; отсутствие изобра-

жения не скрывает название игры. При переходе к внешнему руководству или отзыву ссылка открывается только при допустимой схеме HTTP или HTTPS. Если внешний отзыв не получен, собственные отзывы GameKritik остаются видимыми и доступными для публикации.

1.4 Выбор стека технологий

Для клиентской части выбран Kotlin: проект открывается в Android Studio, а Gradle фиксирует версии Android SDK и библиотек. Интерфейс построен на Jetpack Compose и Material 3. Такой подход позволяет описывать состояние экрана и его отображение в одном стиле, а списки игр выводить с помощью компонентов для ленивой прокрутки.

Удалённое хранение организовано в Supabase на PostgreSQL. Встроенный Supabase Auth выполняет регистрацию и вход, REST API предоставляет данные каталога, а функции PostgreSQL обслуживают личные операции. Для скриншотов используется загрузка изображений по URL, для трейлеров — Media3 ExoPlayer или переход к источнику. Прототипы и визуальная иерархия подготовлены в Figma; версии исходников фиксируются Git и опубликованы на GitHub.

Таблица 1.2 — Выбор технологий разработки

Технология	Назначение	Причина выбора
Kotlin	Логика Android-клиента	Совместимость с Android SDK
Jetpack Compose	Построение экранов	Декларативный интерфейс
Supabase и PostgreSQL	Каталог и личные данные	Auth, REST API и SQL в одном сервисе
Media3 и Coil	Видео и изображения	Поддержка потоков и загрузки по URL
Git и GitHub	Версии и публикация	История изменений и доступ к исходникам

1.5 Архитектура приложения

GameKritik построен по клиент-серверной схеме. Android-клиент отвечает за интерфейс и состояние экранов; Supabase предоставляет каталог через REST API, а Supabase Auth выдаёт токены пользователя. Для личных операций используются функции PostgreSQL, вызываемые через endpoint RPC. Отзывы Steam загружаются отдельным запросом к Steamworks и не проходят через Supabase. Состав компонентов раскрыт ниже.

На стороне Android композиционные функции файла MainActivity.kt формируют экраны, GameViewModel.kt хранит состояние и запускает асинхронные действия, а SupabaseRepository из Data.kt выполняет сетевые запросы и преобразует JSON в модели предметной области. SessionStore сохраняет токены между запусками. Такой порядок разделяет визуальную часть и работу с данными: экран не должен напрямую строить HTTP-запрос или интерпретировать ответ базы. Изменение данных в модели приводит к обновлению соответствующего интерфейса Compose [1]. GameViewModel наследует AndroidViewModel; сетевые операции выполняются в viewModelScope, а наблюдаемое состояние Compose обновляет экран после ответа сервера.

Сценарий открытия игры проходит несколько шагов. Каталог получает набор записей games, пользователь выбирает карточку, после чего экран описания использует уже загруженную модель игры. При переходе к скриншотам, трейлерам, гайдам или отзывам ViewModel запрашивает только соответствующий раздел. Для авторизованного пользователя открытие игры дополнительно вызывает функцию записи истории; повторный просмотр переносит игру в начало списка. Такая загрузка по требованию уменьшает стартовый объём сетевого обмена.

1.6 Проектирование интерфейса в Figma

Эскизы экранов подготовлены в Figma, после чего визуальные решения перенесены в Android. Основные сценарии представлены вкладками «Главная», «Каталог», «Избранное» и «Профиль». В каталоге над списком

расположены поиск, категории и фильтр жанров; карточка показывает широкую обложку и обязательно сохраняет читаемое название под ней. Такой подход устраняет проблему обрезанного логотипа внутри узкого изображения. На странице игры пользователь видит основные сведения и отдельные переходы к отзывам, гайдам, модам и медиа.

В интерфейсе предусмотрены два набора цветов, выбираемые по системной настройке телефона. Светлая тема использует светлые поверхности и тёмный текст, тёмная — тёмные поверхности и светлый текст; красный цвет выделяет активные действия и навигацию. Наличие пустых и ошибочных состояний предусмотрено на этапе проектирования: игра может не иметь трейлера, будущий релиз — модов, а внешний сервис — временно не отвечать. Такие ситуации отображаются текстом и доступным следующим действием вместо пустого экрана.

Макет GameKritik доступен по адресу: https://www.figma.com/design/LQwdQHan4zajmZqupOnG1U/

Для проверки переноса макета на Android сопоставлены приветственный экран, каталог и страница игры в тёмной теме. На рисунке 1.1 приведены кадры исходного проекта Figma.

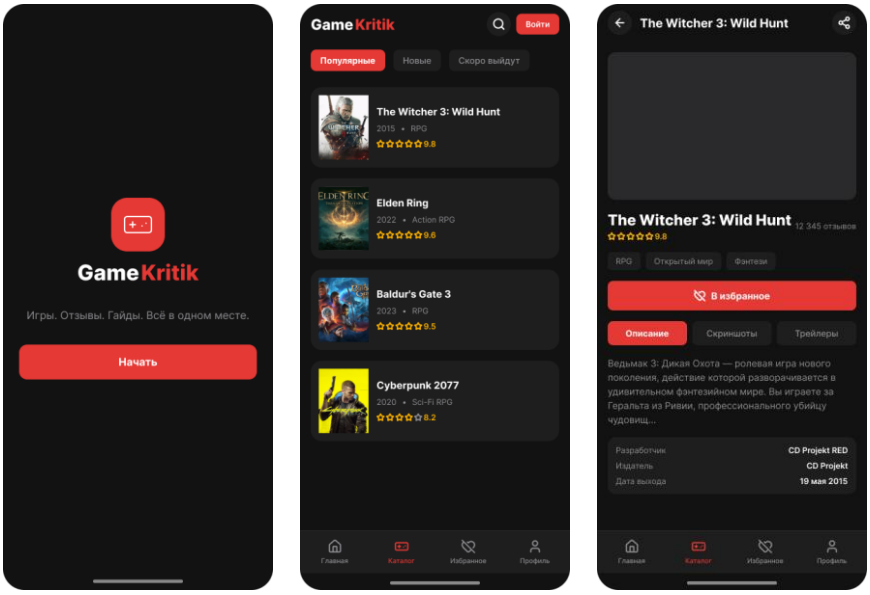


Рисунок 1.1 — Макеты экранов GameKritik в Figma

Реализованные приветственный экран и каталог показаны на рисунке 1.2. В приложении сохранены тёмная палитра, красные элементы управления и расположение основных блоков макета.

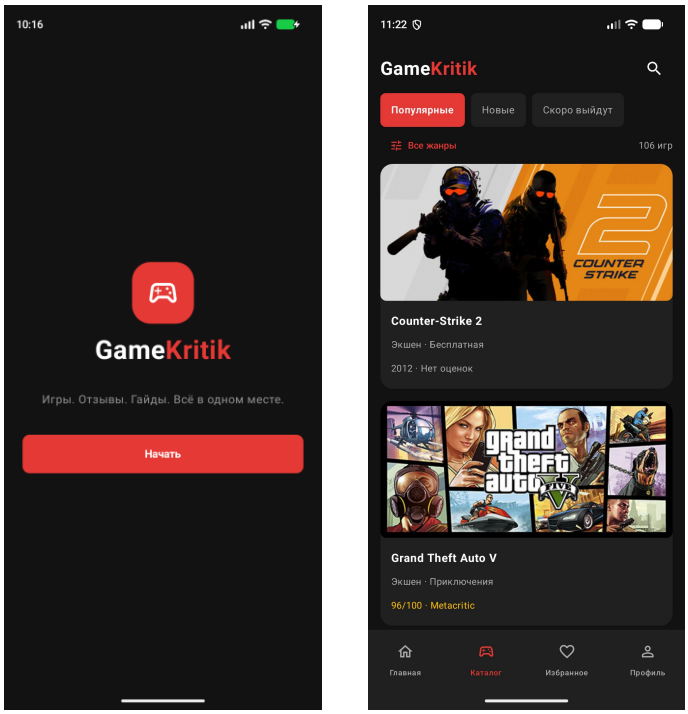


Рисунок 1.2 — Экраны Android-приложения

1.7 Проектирование базы данных в Supabase

В основе хранения лежат связанные таблицы PostgreSQL. Таблица games содержит атрибуты игр; screenshots, trailers, guides и mods связаны с ней по game_id. Учётная запись Supabase Auth связана с прикладным профилем users через auth_id. Таблицы favorites, reviews и game_history содержат одновременно идентификаторы пользователя и игры. Эти связи закреплены в SQL-файлах проекта. Разделение таблиц исключает повторение длинных описаний игры в каждом отзыве и позволяет независимо добавлять материалы.

Публичные сведения о каталоге читаются без входа. Для личных данных применяется иной режим: доступ к таблицам пользователей, избранного и истории ограничен, а операции выполняются через функции gk_profile, gk_favorites, gk_set_favorite, gk_write_review, gk_history, gk_record_view и gk_my_reviews. Функция определяет владельца по токену

текущего запроса, а не по произвольному идентификатору, переданному приложением. Например, история собственных отзывов выбирается по user_id, полученному из профиля авторизованного пользователя. Это снижает риск чтения чужих записей при подмене параметра запроса.

Supabase Auth хранит учётные записи отдельно от прикладных таблиц. Правила Row Level Security и разрешения PostgreSQL вместе определяют допустимые операции: одного включения RLS недостаточно, если роли выданы лишние права [6, 7]. Поэтому для персональных таблиц выполнен отзыв прямых разрешений на чтение и изменение, а для клиентских функций предоставлен только необходимый доступ роли authenticated. Публичный ключ проекта допустим в клиентском приложении; сервисный секрет или пароль базы в APK не размещаются.

2 РЕАЛИЗАЦИЯ МОБИЛЬНОГО ПРИЛОЖЕНИЯ

2.1 Настройка среды разработки и подключение Supabase

Проект создан для Android Studio и собирается Gradle. Исходный код написан на Kotlin; минимальная поддерживаемая версия Android соответствует API 26, целевая и компилируемая конфигурация определены в Gradle. Интерфейс реализован Jetpack Compose с Material 3. Для загрузки изображений используется Coil, а воспроизведение HLS-трейлеров выполняет Media3 ExoPlayer. Эти библиотеки решают разные задачи: Compose описывает дерево экрана, Coil асинхронно получает обложки, Media3 управляет состоянием видеопотока [1, 2, 3].

На этапе подготовки были заданы адрес проекта Supabase и публичный publishable key. Клиент обращается к REST-путям /rest/v1 для каталогов и /auth/v1 для регистрации и входа; обращения к функциям PostgreSQL выполняются через /rest/v1/rpc. Такой способ соответствует открытому Data REST API Supabase [8]. При сетевом запросе клиент устанавливает заголовок apikey, а для личных функций добавляет действующий JWT в Authorization. Секрет администратора в клиент не включён.

2.2 Реализация ключевых экранов приложения

После запуска приложение получает список игр из таблицы games и преобразует ответ JSON в объекты Game. Модель хранит не только название и описание, но и статус релиза, дату первого выхода, жанр, источник обложки и показатель популярности. На основании этих полей функция filterGames формирует категории. «Популярные» сортируются по показателю популярности, «Новые» ограничиваются релизами 2025–2026 годов, «Скоро выйдут» используют статус и дату. Поиск сравнивает название без учёта регистра, а жанр накладывает дополнительное условие.

Отрисовка длинного списка выполняется компонентом LazyColumn, который создаёт видимые карточки по мере прокрутки. Каждая карточка включает широкое изображение Steam и отдельную текстовую строку с названием; изображение выводится целиком с сохранением пропорций. На рисунке 2.1 показан каталог в тёмной теме. В актуальном состоянии список содержит 85 игр; это число получено из облачной базы после сокращения каталога, а не задано константой в интерфейсе.

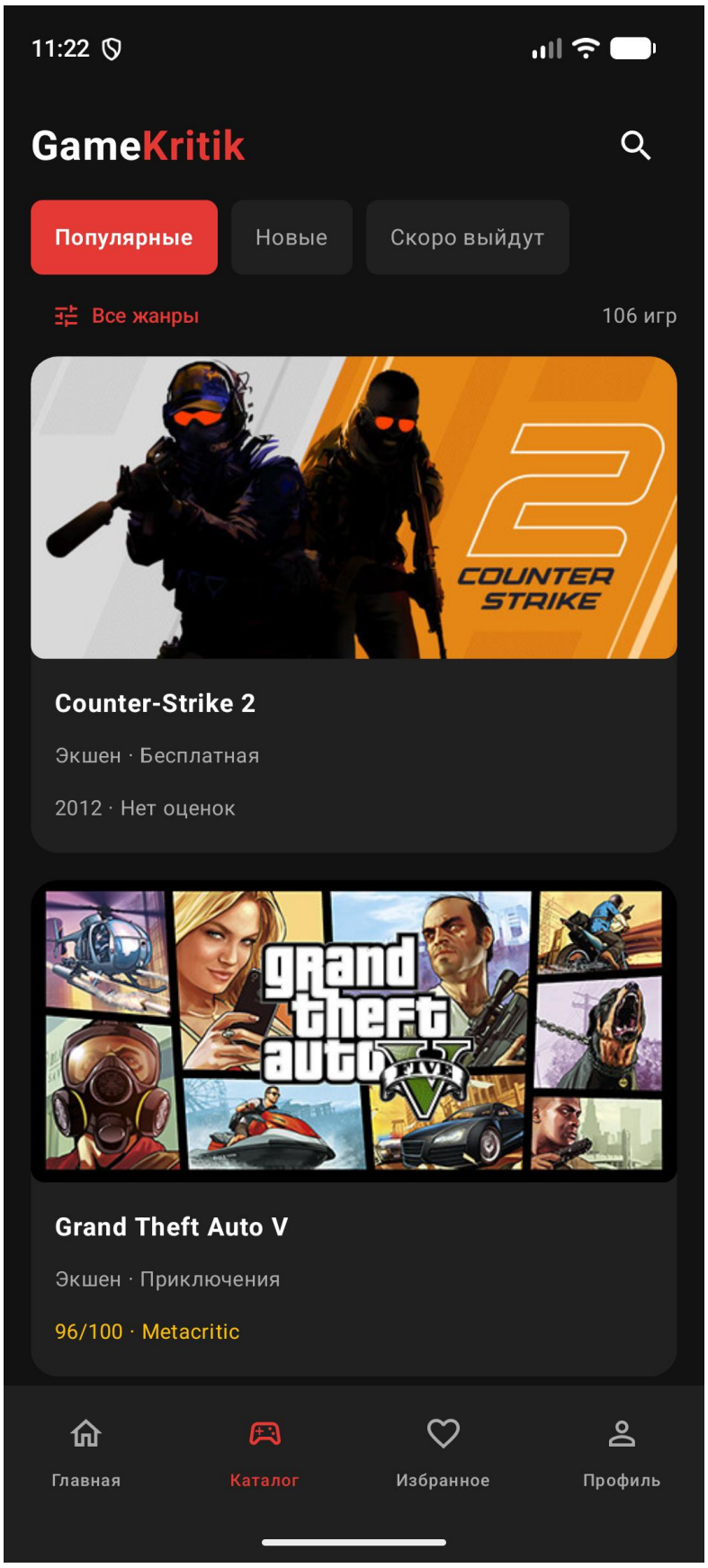


Рисунок 2.1 — Каталог игр в тёмной теме

Нижняя навигация задаёт четыре постоянных направления: главная страница, каталог, избранное и профиль. Выбор игры открывает подробный экран поверх текущего раздела. Системная кнопка «Назад» возвращает пользователя последовательно из материала к странице игры, из подраздела к описанию, а затем к каталогу. Для гостя раздел избранного показывает объяснение и кнопку входа. В результате отсутствие сессии не приводит к пустому списку без причины.

Страница игры объединяет заголовок, обложку, жанр, платформы, разработчика, издателя, дату выхода, оценку и подробное описание. Для ожидаемого релиза вместо пользовательского рейтинга акцентируется дата выхода. Кнопка избранного меняет состояние только после успешного обращения к защищённой функции Supabase. Отдельные кнопки ведут к отзывам, гайдам и модам; вкладки открывают скриншоты и трейлеры. Разделы загружаются при выборе, поэтому медиа не замедляют первоначальный показ описания.

Скриншоты поступают из таблицы screenshots и отображаются в галерее. При нажатии открывается горизонтальный полноэкранный просмотр с сохранением пропорций изображения. Видеоматериалы получаются из trailers и воспроизводятся Media3 ExoPlayer, если источник предоставляет поддерживаемый HLS или MP4-поток. Если поток не готов или URL не поддерживается встроенным плеером, остаётся возможность открыть страницу источника. Исходные ссылки на изображения и трейлеры сохраняются в базе без преобразования содержимого.

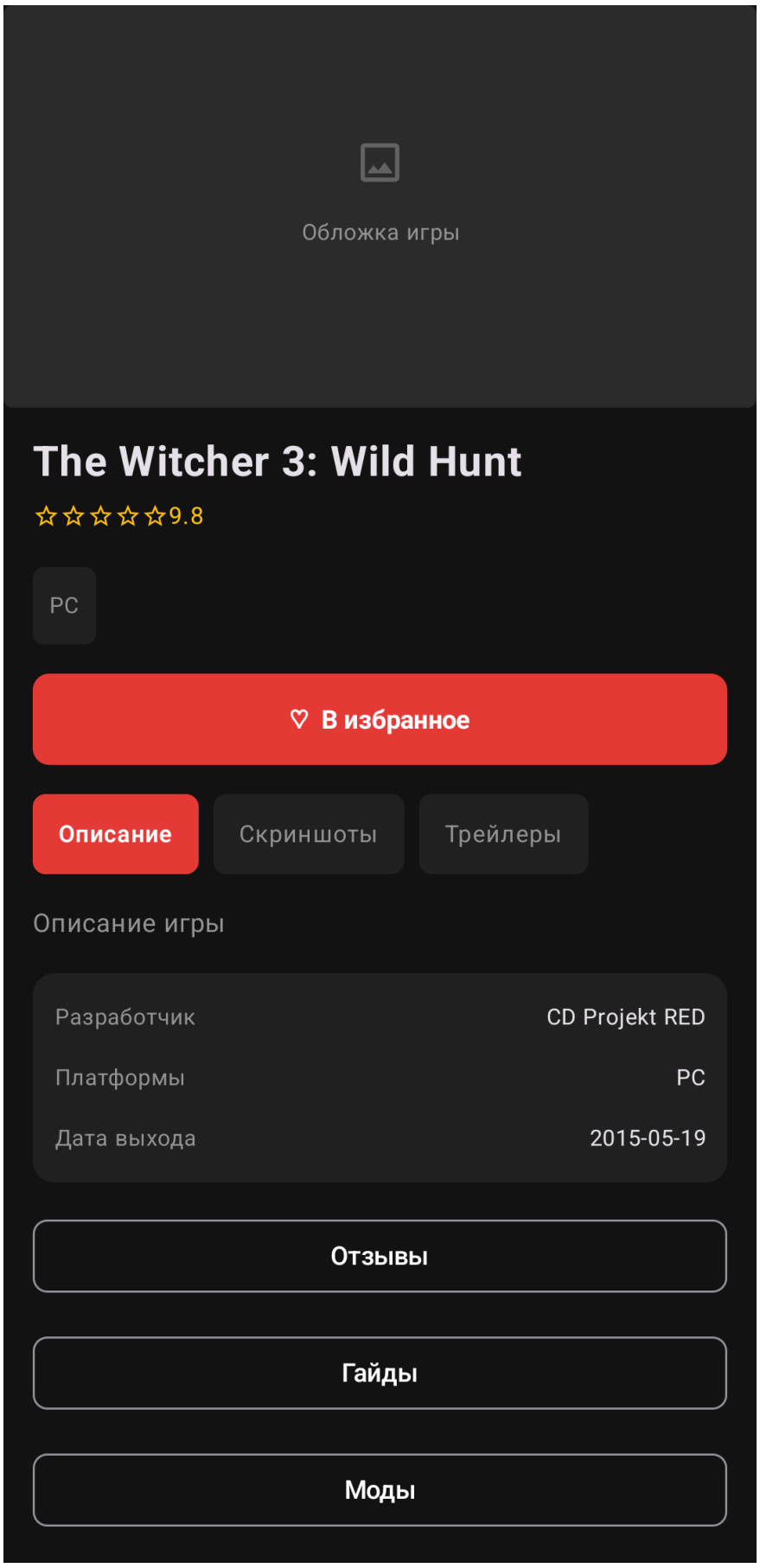


Рисунок 2.2 — Страница игры в тёмной теме

Состояние каталога и выполнение сетевой загрузки вынесены в GameViewModel. Функция refresh запускает запрос в viewModelScope и обновляет наблюдаемое состояние для Compose (рисунок 2.3).

GameViewModel.kt

```
45      fun refresh() {
46          if (loading) return
47          viewModelScope.launch {
48              loading = true
49              try { games = repository.games() } catch (e: Exception) { if (e is
    CancellationException) throw e; failure(e) }
50              finally { loading = false }
51          }
52      }
53      private fun action(block: suspend () -> Unit) {
54          if (busy) return
55          viewModelScope.launch {
56              busy = true; error = null
57              try { block() } catch (e: Exception) { if (e is CancellationException)
    throw e; failure(e) }
58              finally { busy = false }
59          }
60      }
```

Рисунок 2.3 — Загрузка каталога и обработка действий в GameViewModel.kt

2.3 Взаимодействие с базой данных Supabase

Регистрация принимает никнейм, адрес электронной почты и пароль. Клиент проверяет формат адреса, минимальную длину пароля и совпадение двух вводов перед отправкой формы. Supabase Auth создаёт учётную запись в auth.users, после чего триггер формирует прикладной профиль в public.users. При входе сервер выдаёт access и refresh tokens; при истечении срока действия access token клиент обновляет его через refresh token. Пароль после отправки формы не хранится в памяти постоянного профиля и не записывается в локальную базу приложения.

SessionStore защищает сохранённые токены механизмом Android Keystore и шифрованием AES-GCM. При повторном запуске приложение восстанавливает сессию, загружает профиль, избранное, просмотренные игры и собственные отзывы. Выход очищает локальную сессию и личные данные ViewModel. Для учебной версии подтверждение электронной почты было отключено по решению владельца проекта, поэтому при даль-

нейшей публикации необходимо вернуть проверку адреса и настроить доставку писем.

Профиль показывает никнейм, число избранных игр, карточки собственных отзывов с оценкой и датой, а также историю посещений. Нажатие на отзыв открывает раздел отзывов соответствующей игры; нажатие на просмотренную игру открывает её описание. История хранится в таблице game_history, обновляется при открытии игры и ограничена 30 последними записями на пользователя. Список собственных отзывов формирует защищённая функция gk_my_reviews, которая выбирает записи по идентификатору текущего авторизованного аккаунта. Собственный отзыв можно изменить на странице игры или удалить из профиля после подтверждения; сервер проверяет владельца записи и пересчитывает оценку игры.

В сетевом слое запрос выполняется с таймаутом соединения и чтения. Ошибка HTTP преобразуется в понятное сообщение: неверный пароль, неподтверждённая почта, превышение лимита запросов или отсутствие доступа различаются в интерфейсе. Действия с личными данными требуют JWT и не выполняются под гостевой ролью. В то же время публичные сведения каталога доступны без входа. Такое разделение соответствует правилам Supabase Auth и REST API, в которых токен определяет роль и область доступа запроса [6, 8].

Безопасность обеспечивается не только проверками на телефоне. Функции gk_write_review и gk_my_reviews определяют владельца на сервере, а изменения избранного и истории не принимают внешний user_id как доверенный параметр. Для game_history включён RLS и отозван прямой доступ клиентских ролей. В приложении сохранён лишь публичный ключ проекта, а локальные токены шифруются. При публикации проекта следует дополнительно провести аудит политик, вернуть подтверждение почты и исключить демонстрационные оценки, оставшиеся от учебного макета. Функция gk_delete_review также определяет владельца по JWT и не принимает внешний идентификатор пользователя.

Взаимодействие с Supabase Auth реализовано в SupabaseRepository. Фрагмент Data.kt на рисунке 2.4 показывает запросы для входа и регистрации и сохранение полученной сессии.

Data.kt

```
90      suspend fun login(email: String, password: String) {
91          acceptSession(JSONObject(request("/auth/v1/token?grant_type=password",
        "POST", JSONObject().put("email", email.trim()).put("password", password))))
92      }
93      suspend fun register(name: String, email: String, password: String): Boolean {
94          logout()
95          val response = JSONObject(request("/auth/v1/signup", "POST",
        JSONObject().put("email", email.trim()).put("password", password).put("data",
        JSONObject().put("name", name.trim()))))
96          acceptSession(response)
97          return session != null
98      }
```

Рисунок 2.4 — Вход и регистрация через Supabase Auth в Data.kt

2.4 Реализация дополнительного функционала

Для каждой игры в каталоге существует раздел руководств. Карточки группируются по назначению: прохождения, достижения и советы; нажатие открывает описание и ссылку на тематическую подборку сообщества Steam. Тексты сторонних полных руководств не копируются в приложение. Для ожидаемых проектов размещается подготовительная информация до выхода игры. Такой формат сохраняет полезный путь к материалу и указывает его происхождение.

Раздел модов заполнен карточками со ссылками на Steam Community и, для ряда игр, Nexus Mods. В карточке приводятся назначение страницы и рекомендации проверить совместимость с версией игры. Файлы модификаций не скачиваются и не распространяются через GameKritik. Для будущих релизов показано сообщение, что подтверждённых модов ещё нет. Карточка игры и переход к внешнему ресурсу остаются частью одной экранной иерархии.

В разделе отзывов собственные записи GameKritik отделены от внешних отзывов Steam. Приложение запрашивает до 100 русскоязычных записей через метод Steamworks, затем выбирает до пяти записей с высо-

ким числом отметок «забавно». Карточка показывает признак рекомендации, короткий текст, дату, счётчик и ссылку на оригинал. При недоступности Steam пользователь видит сообщение и ссылку на страницу игры. API возвращает данные динамически; их тексты не сохраняются в Supabase. Это обеспечивает актуальность и сохраняет ясное указание источника [9].

Для проверки использовались сборка Gradle, UI-тесты Compose и запуск на эмуляторе Pixel 8. Сборка assembleDebug подтверждает корректность исходного кода и ресурсов, assembleDebugAndroidTest — компиляцию тестового APK. Обычные экранные проверки контролируют переход с приветственного экрана, открытие страницы игры, запрет отправки слишком короткого отзыва и полноэкранный просмотр медиа. Отдельный тест обращается к действующему облачному каталогу и проверяет количество игр, обложку, наличие материала, трейлер и навигацию. На физическом телефоне Android 16 выполнен сценарий DefenseDemoTest: регистрация, создание, изменение и удаление отзыва, выход и повторный вход. Проверка завершилась результатом OK (1 test); запись экрана и журнал размещены в репозитории проекта.

Таблица 2.1 — Результаты функциональной проверки

Сценарий	**Ожидаемый результат**	**Результат**
Поиск игры	Карточки фильтруются по названию	Соответствует
Регистрация и вход	Создаётся пользователь, восстанавливается сессия	Соответствует
Избранное	Список сохраняется после перезапуска	Соответствует
Отзыв	Запись появляется в игре и профиле	Соответствует
Медиа	Скриншот и трейлер открываются на весь экран	Соответствует
Системная тема	Оформление следует теме Android	Соответствует
Ошибка сети	Показывается сообщение, экран остаётся доступен	Соответствует

Проекту потребуется регулярное редакционное сопровождение: уточнение дат будущих игр, обновление обложек и источников трейлеров, проверка ссылок на модификации и руководства. Данные каталога отделены от кода экранов, поэтому добавление карточки в Supabase не требует новой логики навигации. При этом редактор должен проверять действительность внешнего URL и содержание материала. Некорректная ссылка

ухудшает пользовательский сценарий даже тогда, когда приложение технически работает без ошибок.

2.5 Работа с Git

Для контроля изменений использована система Git. Публичный репозиторий GameKritik содержит полный проект Android Studio: исходники модуля app, конфигурацию Gradle и wrapper, SQL-миграции и исходные данные в каталоге supabase, скриншоты, README, архив исходников и установочный APK. Адрес репозитория: https://github.com/vlad2149/GameKritik.

Локальная копия связана с удалённой веткой main. После проверки файлов изменения фиксировались коммитом и отправлялись на GitHub. Сборочные каталоги, локальные настройки Android SDK и файлы среды разработки исключены правилами .gitignore. Это позволяет скачать исходники и открыть корневую папку проекта в Android Studio без переноса машинно-зависимых путей.

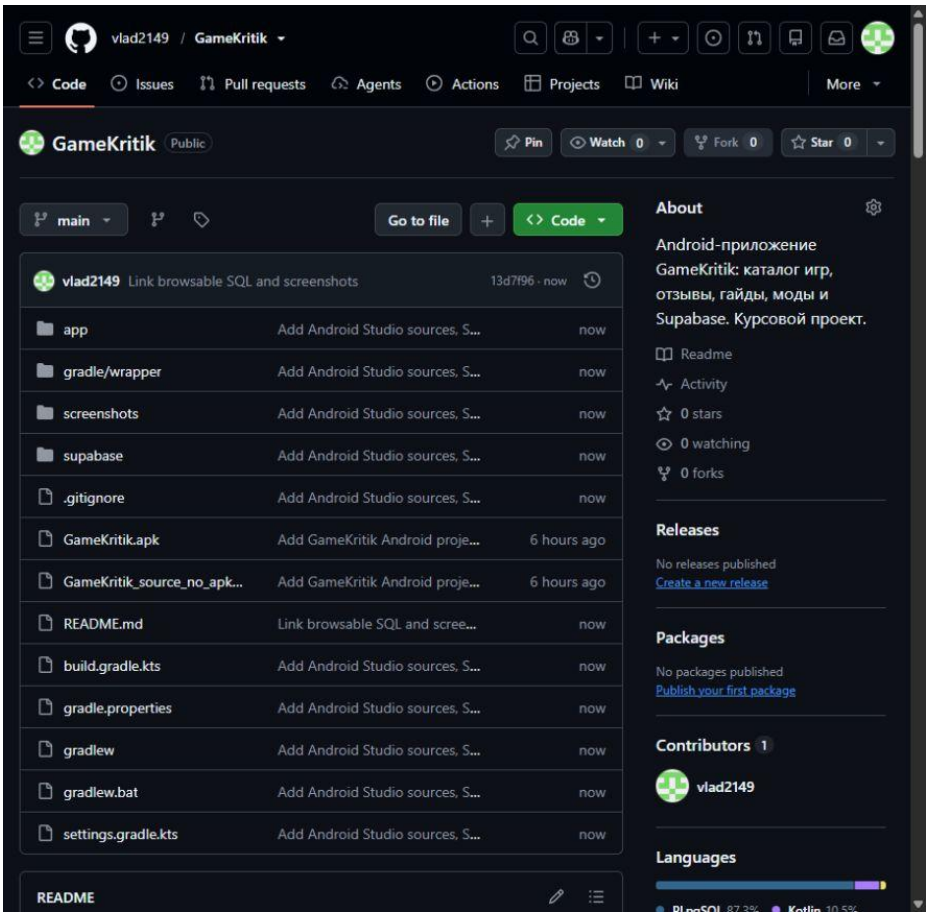


Рисунок 2.5 — Репозиторий GameKritik на GitHub

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы спроектировано и реализовано Android-приложение GameKritik, объединяющее каталог игр, медиа, руководства, модификации, пользовательские отзывы и личную историю. Каталог из 85 игр хранится в Supabase и обновляется без пересборки интерфейса. Регистрация и вход выполняются через Supabase Auth, а защищённые функции PostgreSQL возвращают только данные владельца текущей сессии. Задачи, поставленные во введении, выполнены: структура приложения спроектирована, экранные формы реализованы, база подключена, сборка и ключевые сценарии проверены.

Практическая ценность результата состоит в едином мобильном сценарии: пользователь находит игру, изучает описание, смотрит трейлер и скриншоты, переходит к гайдам и модам, читает отзывы Steam, публикует собственный отзыв и видит его в профиле. При этом приложение различает собственные и внешние материалы, а персональные действия сохраняет в облачной базе. Дизайн автоматически подстраивается под светлую или тёмную тему телефона.

Дальнейшее развитие связано с редакционным обновлением каталога, проверкой дат будущих релизов, расширением покрытия реальных гайдов и модификаций, настройкой подтверждения почты для публичного запуска и более широким тестированием на устройствах. Для внешних ресурсов следует сохранять ссылку на оригинал и учитывать доступность сервиса. Созданная архитектура допускает добавление новых игр и материалов без изменения основных экранов и функций авторизации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Android Developers. Jetpack Compose : документация [Электронный ресурс]. — URL: https://developer.android.com/compose (дата обращения: 21.10.2026).
2. Android Developers. Media3 ExoPlayer : документация [Электронный ресурс]. — URL: https://developer.android.com/media/media3/exoplayer (дата обращения: 21.10.2026).
3. Coil. Compose : документация [Электронный ресурс]. — URL: https://coil-kt.github.io/coil/compose/ (дата обращения: 21.10.2026).
4. Figma Help Center. Guide to prototyping in Figma [Электронный ресурс]. — URL: https://help.figma.com/hc/en-us/articles/360040314193-Guide-to-prototyping-in-Figma (дата обращения: 21.10.2026).
5. Kotlin. Documentation [Электронный ресурс]. — URL: https://kotlinlang.org/docs/home.html (дата обращения: 21.10.2026).
6. Supabase Docs. Auth [Электронный ресурс]. — URL: https://supabase.com/docs/guides/auth (дата обращения: 21.10.2026).
7. Supabase Docs. Row Level Security [Электронный ресурс]. — URL: https://supabase.com/docs/guides/database/postgres/row-level-security (дата обращения: 21.10.2026).
8. Supabase Docs. Data REST API [Электронный ресурс]. — URL: https://supabase.com/docs/guides/api (дата обращения: 21.10.2026).
9. Steamworks Documentation. User Reviews — Get List [Электронный ресурс]. — URL: https://partner.steamgames.com/doc/store/getreviews (дата обращения: 21.10.2026).
10. Саввина Е. В. Методические указания по выполнению курсовой работы по МДК 01.03 «Разработка мобильных приложений». — Якутск, 2025.
11. GameKritik: исходный код Android-приложения и материалы проекта [Электронный ресурс]. — URL: https://github.com/vlad2149/GameKritik (дата обращения: 28.10.2026).

ПРИЛОЖЕНИЕ А

Проверка каталога в базе данных Supabase

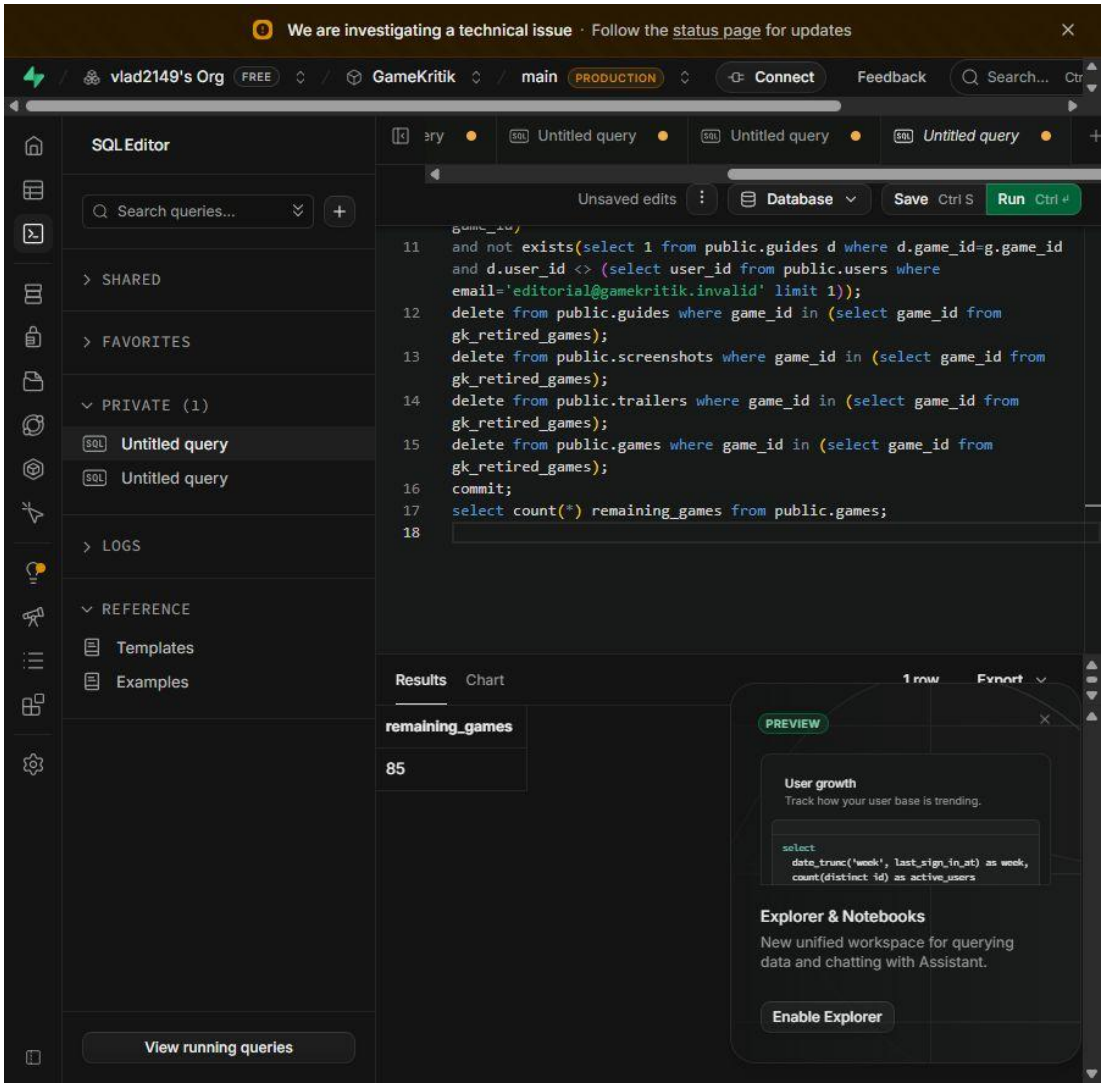


Рисунок А.1 — Проверка записей каталога в Supabase